Práctica 4: Proceso de Arranque (Boot)

Belmont Muñoz Samuel
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 317318289

Domínguez Palacios Jesús Alejandro
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 421066205

Dorantes Mejía Emmanuel
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 313004766

García Olvera Emiliano
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 314256009

Hernández Alonso Antonio Adonahí
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 320257780

Jiménez Reyes Alan Yael
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 320026726

Práctica No.: 4 | **Grupo:** 6 | **Fecha:** DD/MM/AAAA

***Resumen*—Este reporte documenta el análisis del proceso de arranque (boot) de la Raspberry Pi 4, desde el bootloader almacenado en EEPROM hasta la inicialización completa del sistema operativo bajo el control de systemd. Se describe la identificación de las etapas del arranque, la inspección del bootloader, el análisis de logs con `journalctl`, la medición de tiempos de arranque con `systemd-analyze`, y la creación de servicios personalizados. El documento aborda técnicas de optimización del tiempo de arranque y su medición cuantitativa, demostrando cómo systemd gestiona la inicialización del sistema en Linux embebido.**

***Index Terms*—Boot, EEPROM, bootloader, systemd, journalctl, systemd-analyze, servicios, Raspberry Pi, init system**

I. Objetivo

I-A. Objetivo General

Comprender las etapas del proceso de arranque de la Raspberry Pi, desde el bootloader en EEPROM hasta el sistema completamente iniciado, utilizando las herramientas de diagnóstico de systemd para inspeccionar, medir y modificar el proceso de arranque.

I-B. Objetivos Específicos

- Identificar las etapas del arranque de la Raspberry Pi, del bootloader al sistema completo.
- Verificar la versión del bootloader (EEPROM) sin necesidad de actualizarlo.
- Inspeccionar el log completo de arranque con `journalctl`.
- Medir y analizar el tiempo de arranque con `systemd-analyze`.
- Crear un servicio systemd propio que arranque automáticamente uno de los programas desarrollados en prácticas anteriores.
- Optimizar el tiempo de arranque, midiendo el efecto real de los cambios aplicados.
- Registrar los avances en el repositorio de Git del curso.

II. Introducción

El proceso de arranque de un sistema embebido es una secuencia compleja de etapas que transforman un hardware inerte en un sistema operativo funcional. En la Raspberry Pi 4, este proceso involucra múltiples componentes: el bootloader almacenado en EEPROM, el firmware de la GPU, el kernel de Linux, y el sistema de inicialización systemd.

II-A. El Bootloader en EEPROM

A diferencia de computadoras tradicionales que utilizan BIOS o UEFI almacenadas en la placa madre, la Raspberry Pi 4 almacena su bootloader en un chip EEPROM integrado en la placa. Este bootloader es lo primero que se ejecuta al aplicar energía al sistema y se encarga de localizar y cargar el siguiente componente del proceso de arranque desde la tarjeta microSD.

El bootloader en EEPROM puede actualizarse mediante el sistema operativo en ejecución, permitiendo correcciones de errores y nuevas características sin reemplazar hardware. La versión del bootloader determina capacidades como el arranque desde USB, configuraciones de red PXE, y la gestión de energía durante el arranque.

II-B. Firmware de la GPU y Device Tree

El segundo componente del arranque es el firmware de la GPU (VideoCore IV), almacenado en archivos como `start4.elf` en la partición boot de la microSD. Este firmware lee el archivo de configuración `config.txt`, inicializa el hardware básico (RAM, reloj del sistema, periféricos), y carga el kernel de Linux junto con el Device Tree.

El archivo `config.txt` permite configurar parámetros de hardware antes de que Linux arranque, como la habilitación de interfaces I2C y SPI (utilizada en la Práctica 3), overclocking, asignación de memoria a la GPU, y configuración de salidas de video. Esta etapa es fundamental porque establece el estado inicial del hardware que el kernel asumirá al tomar control.

II-C. Kernel de Linux

El kernel de Linux, usualmente almacenado como `kernel8.img` para arquitecturas de 64 bits, toma control

del procesador ARM después de ser cargado por el firmware de la GPU. El kernel inicializa los subsistemas de gestión de memoria, scheduling de procesos, y drivers de dispositivos, monta el sistema de archivos raíz, y finalmente ejecuta el primer proceso de espacio de usuario: systemd (PID 1).

II-D. systemd como Sistema de Inicialización

systemd es el sistema de inicialización (init system) utilizado por la mayoría de distribuciones modernas de Linux, incluyendo Raspberry Pi OS. A diferencia de sistemas de init tradicionales basados en scripts secuenciales, systemd utiliza un modelo de dependencias paralelas que permite arrancar servicios simultáneamente cuando sus dependencias están satisfechas.

systemd gestiona servicios mediante archivos de unidad (*unit files*) que definen cómo y cuándo deben iniciarse los servicios. Proporciona herramientas poderosas para el diagnóstico del arranque:

- `journalctl` — Sistema de logging centralizado que captura todos los mensajes del kernel y servicios.
- `systemd-analyze` — Herramienta para medir y visualizar el tiempo de arranque de cada servicio.
- `systemctl` — Interfaz para gestionar servicios, habilitar/deshabilitar su arranque automático, y consultar su estado.

II-E. Etapas del Arranque de la Raspberry Pi

La tabla I resume las cuatro etapas principales del proceso de arranque:

Cuadro I
ETAPAS DEL ARRANQUE DE LA RASPBERRY PI.

#	**Etapa**
1	Bootloader (EEPROM): busca firmware en microSD.
2	Firmware GPU: lee `config.txt`, carga kernel.
3	Kernel: inicializa SO, monta raíz, ejecuta systemd.
4	systemd: arranca servicios según dependencias.

II-F. Relación con Prácticas Anteriores

En la Práctica 3, se modificó el archivo `/boot/firmware/config.txt` para habilitar las interfaces I2C y SPI mediante las líneas:

```
dtparam=i2c_arm=on
dtparam=spi=on
```

Esta configuración no activa los buses "mágicamente" después de que Linux arranca, sino que modifica la Etapa 2 del arranque: el firmware de la GPU lee estas directivas y configura el hardware correspondiente antes de cargar el kernel. Linux hereda esta configuración y hace disponibles las interfaces `/dev/i2c-1` y `/dev/spidev0.0` durante su inicialización.

Esta práctica profundiza en el entendimiento de todo el proceso de arranque, permitiendo diagnosticar problemas, optimizar tiempos de inicio, y crear servicios personalizados que se ejecuten automáticamente al encender el sistema.

III. DESARROLLO

III-A. Material y Equipo

Para la realización de esta práctica no se requieren componentes de hardware adicionales. Se utilizó únicamente:

- Raspberry Pi 4 Model B con Raspberry Pi OS instalado (configurada en prácticas anteriores)
- Laptop con acceso SSH al sistema
- Toolchain de desarrollo en C (GCC) previamente instalado
- Repositorio Git del curso con prácticas anteriores
- Programa `sensor_combinado.c` de la Práctica 3 (o alternativamente `leds.c` de la Práctica 2)

III-B. Parte A — Confirmar las etapas en la Raspberry Pi

Se estableció conexión por SSH con la Raspberry Pi. Para confirmar las etapas iniciales del arranque, se verificó la versión del kernel de Linux y la fecha y hora exacta del arranque mediante los comandos `uname -a` y `uptime -s`.

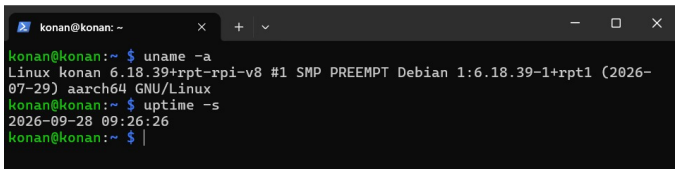


Figura 1. Resultados de la versión del kernel y el tiempo de arranque del sistema.

Posteriormente, se exploró el directorio `/boot/firmware/` donde se identificaron los archivos correspondientes al firmware de la GPU y al kernel (`start4.elf`, `fixup4.dat`, `kernel8.img`, `config.txt` y `cmdline.txt`). Asimismo, se inspeccionó el log del kernel con `dmesg | grep -i i2c` y `dmesg | grep -i spi` para confirmar el momento exacto en que se cargaron los controladores habilitados previamente.

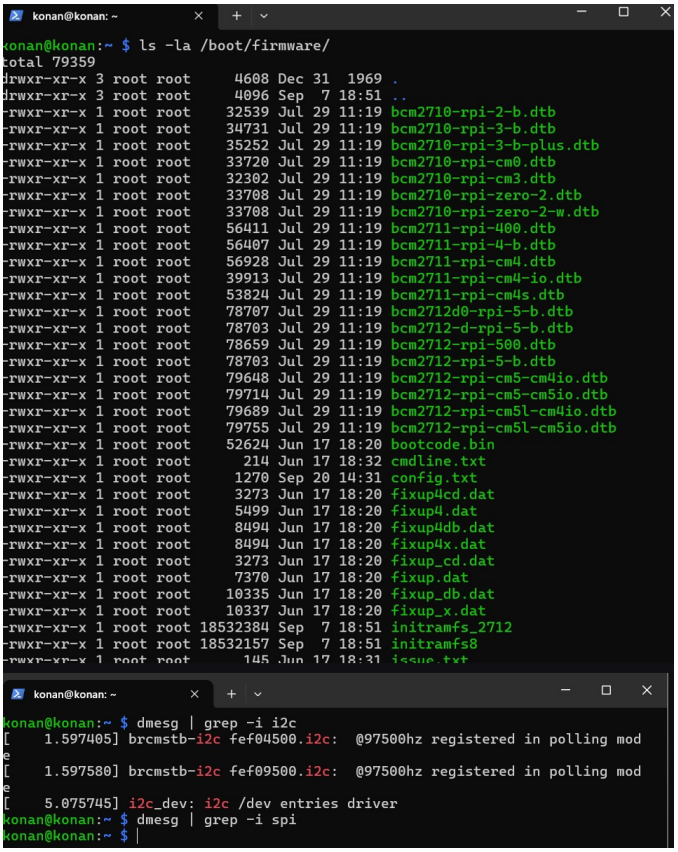


Figura 2. Archivos en la partición de firmware y registros del kernel mostrando la inicialización de I2C y SPI.

III-C. Parte B — Verificar el bootloader y EEPROM

Se consultó la versión actual del bootloader instalado en la EEPROM mediante el comando `vcgencmd bootloader_version`. Además, se verificó si existía alguna actualización disponible ejecutando `sudo rpi-eeprom-update`, aunque no se aplicó ninguna actualización para evitar riesgos de dejar la placa sin arrancar, cumpliendo con el objetivo de únicamente reconocer el firmware.

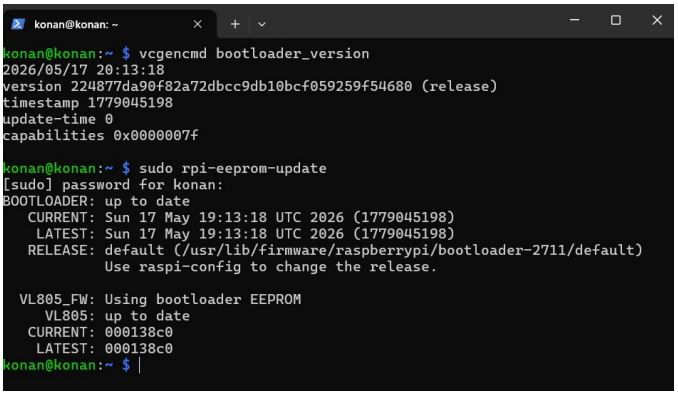


Figura 3. Consulta de la versión del bootloader en la EEPROM y disponibilidad de actualizaciones.

III-D. Parte C — Análisis del log completo de arranque

Se inspeccionó el registro del proceso de arranque utilizando `journalctl -b`. Para facilitar su lectura, se filtró la salida mostrando únicamente advertencias y errores con el comando `journalctl -b -p warning -no-pager`. Finalmente, se revisó si algún servicio había fallado durante el inicio usando `systemctl -failed`, comprobando que se trató de un arranque limpio.

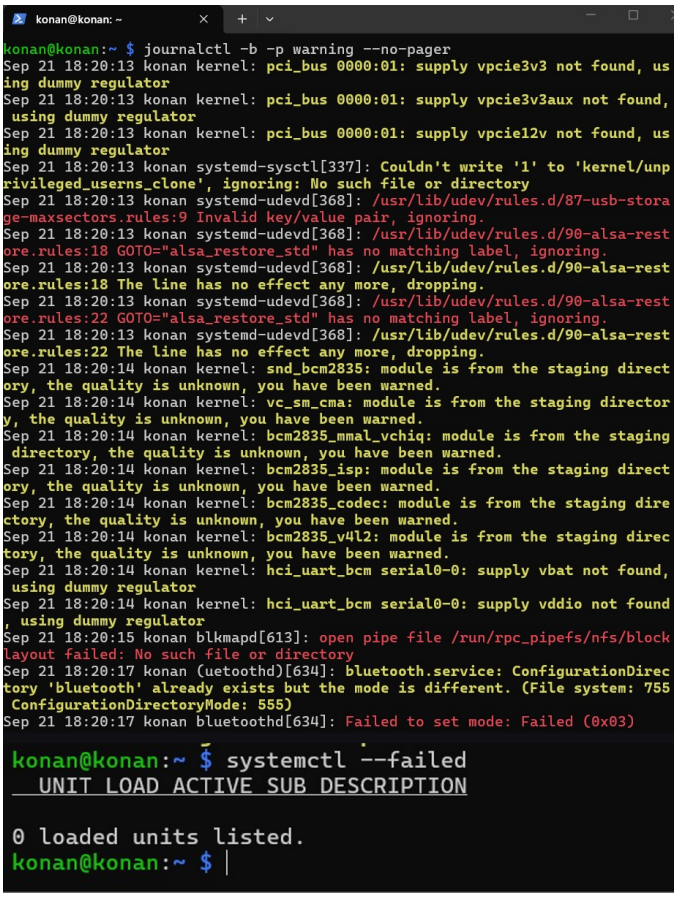


Figura 4. Filtro de advertencias en el log de arranque y verificación de servicios fallidos.

III-E. Parte D — Medición del tiempo de arranque

Para analizar el rendimiento durante el inicio, se midió el tiempo total de arranque con el comando `systemd-analyze`.

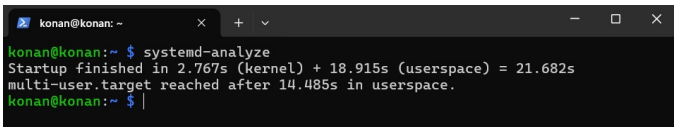


Figura 5. Resumen general del tiempo de arranque del sistema operativo.

Se desglosaron los tiempos individuales de cada servicio mediante `systemd-analyze blame` y se visualizó la cadena crítica de dependencias que determina la du-

ración total del arranque utilizando `systemd-analyze critical-chain`.

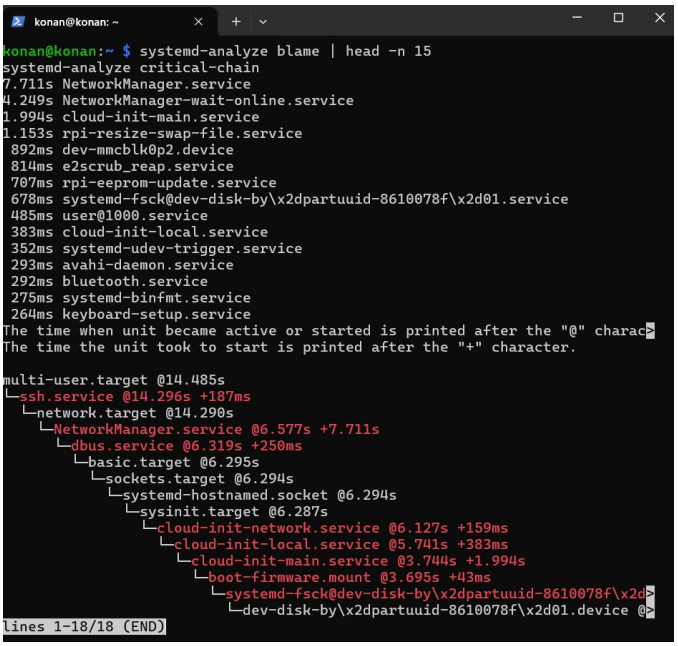


Figura 6. Análisis de los servicios más lentos y la cadena crítica de arranque.

III-F. Parte E — Creación de un servicio de arranque

Se procedió a crear un servicio propio para ejecutar el programa `sensor_combinado` elaborado en prácticas anteriores. Tras confirmar la ruta del ejecutable, se creó el archivo de definición en `/etc/systemd/system/misistema.service`.
El servicio se configuró como tipo `simple` con reinicio en caso de falla (`Restart=on-failure`).

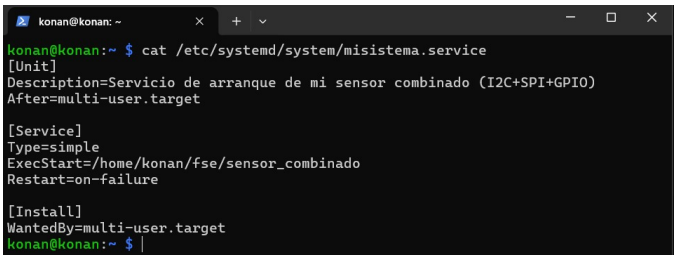


Figura 7. Definición del archivo de configuración para el servicio personalizado misistema.service.

Tras recargar el daemon con `sudo systemctl daemon-reload` y habilitar el servicio (`systemctl enable`), se reinició el sistema y se comprobó que el programa corriera automáticamente verificando su estado con `systemctl status misistema.service`.

¿

Figura 8. Verificación del estado del servicio en ejecución tras reiniciar el sistema.

III-G. Parte F — Optimización del tiempo de arranque

Al analizar la cadena crítica expuesta en la Parte D, se identificó que la carga del entorno gráfico no era indispensable para el trabajo en el laboratorio mediante SSH. Por lo tanto, se modificó la dependencia del sistema removiendo el `graphical.target` (sustituyéndolo por `multi-user.target`) y se deshabilitó explícitamente el gestor de sesiones gráficas con el comando `sudo systemctl disable lightdm`. Tras aplicar estos cambios, se reinició la Raspberry Pi y se midió de nuevo el tiempo con `systemd-analyze`, logrando una reducción y optimización considerable al eliminar estos servicios de la cadena crítica.

Figura 9. Comparación de los tiempos de arranque tras la deshabilitación del target gráfico y lightdm.

III-H. Partes G y H — Control de versiones y detención del servicio

Se respaldó la configuración copiando el archivo `misistema.service` al repositorio local de Git. Se agregaron los cambios, se realizó un commit descriptivo y se comprobó el historial con `git log -oneline`.

Finalmente, para evitar interferencias en futuras sesiones, el servicio fue detenido en la sesión actual mediante `sudo systemctl stop misistema.service` y se deshabilitó su arranque automático de raíz con `sudo systemctl disable misistema.service`, previniendo que los pines GPIO, I2C y SPI quedaran ocupados permanentemente.

Figura 10. Registro del commit en Git y comandos de detención y deshabilitación del servicio.

IV. Resultados y Análisis

Tras la configuración del servicio personalizado, se reinició la Raspberry Pi para comprobar su ejecución automática desde el arranque. Se verificó exitosamente que el programa inició su operación de manera autónoma al encender la placa, lo cual se evidenció físicamente mediante el encendido de un LED rojo (cuya intensidad era controlada proporcionalmente por el programa configurado en prácticas anteriores). Posteriormente, se detuvo y deshabilitó el servicio mediante los comandos correspondientes de `systemctl`, confirmando que el proceso finalizó correctamente, el LED se apagó y el programa dejó de ejecutarse en segundo plano, liberando los pines y recursos del sistema.

Además, respecto a la optimización del tiempo de arranque, se ejecutó nuevamente el comando `systemd-analyze critical-chain` tras aplicar los cambios en la configuración. Se corroboró de forma analítica que los servicios deshabilitados, correspondientes al entorno gráfico y al gestor de sesiones `lightdm`, ya no figuraban como activos ni bloqueaban la secuencia de inicio. Esto confirmó que la exclusión de dichos procesos liberó carga de trabajo y tiempo de arranque (cerca de 7 segundos) del sistema operativo durante

el arranque, reflejándose en una inicialización más eficiente y rápida, ideal para el trabajo en el laboratorio mediante consola por SSH.

V. Conclusiones

Belmont Muñoz Samuel.
A través del desarrollo de esta práctica, se logró comprender de manera integral la secuencia de arranque en la Raspberry Pi, desde las etapas iniciales de hardware y firmware en la EEPROM hasta la inicialización del espacio de usuario gestionada por `systemd`. La inspección de logs y la medición de tiempos permitieron constatar cuantitativamente el impacto que tienen los distintos servicios en la duración total del inicio. Asimismo, la creación de un servicio personalizado demostró la utilidad de `systemd` para la automatización de procesos en sistemas embebidos, garantizando una correcta administración de recursos mediante su posterior detención y deshabilitación. Adicionalmente, la optimización realizada al remover dependencias no esenciales, como el entorno gráfico y el gestor `lightdm`, evidenció cómo es posible adaptar y agilizar el sistema operativo para entornos puramente orientados a tareas de control e ingeniería.

Domínguez Palacios Jesús Alejandro.
Esta práctica me dejó claro que el arranque de la Raspberry Pi no es una caja negra, sino una cadena de cuatro etapas perfectamente identificables: bootloader en EEPROM, firmware de la GPU, kernel y systemd. Verificar con `vcgencmd bootloader_version` que la EEPROM estaba al día y confirmar con `rpi-eeprom-update` que no había actualizaciones pendientes me hizo ver que incluso el primer eslabón de esa cadena es firmware versionado, igual que libgpiod en la Práctica 2: puede quedar desactualizado sin que uno lo note si nunca se revisa. Con `systemd-analyze` medí un arranque de 21.682 s (2.767 s de kernel más 18.915 s de espacio de usuario), y con `blame` y `critical-chain` identifiqué que `NetworkManager.service` era el verdadero cuello de botella de la cadena crítica, no necesariamente el servicio más lento en aislado, lo cual reforzó por qué ambas herramientas hay que revisarlas juntas antes de decidir qué deshabilitar. `journalctl -b -p warning` y `systemctl -failed` confirmaron que, pese a las advertencias normales de hardware no presente, el arranque fue limpio y sin servicios fallidos. Crear `misistema.service` para levantar automáticamente `sensor_combinado.c` cerró el círculo de las prácticas de I2C y SPI: lo que en la Práctica 3 configuramos manualmente en `config.txt` (Etapa 2) ahora arranca solo, sin necesidad de conectarse por SSH, gracias a un archivo de unidad que actúa en la Etapa 4. Por último, la parte de optimización y el cierre de la práctica reforzaron algo muy práctico: un servicio con `Restart=on-failure` que retiene pines GPIO, I2C y SPI hay que detenerlo y deshabilitarlo explícitamente al terminar, porque de lo contrario se convierte en un problema silencioso para la siguiente práctica.

Dorantes Mejía Emmanuel.
Durante esta práctica comprendí de manera más clara el proceso de arranque de la Raspberry Pi, desde el bootloader hasta la inicialización de servicios mediante systemd, además de utilizar herramientas como `systemd-analyze`, `journalctl` y `systemctl` para analizar los tiempos de inicio y verificar el funcionamiento del sistema. También pude relacionar las configuraciones de interfaces como I2C y SPI realizadas anteriormente con las diferentes etapas del arranque. Uno de los aspectos más importantes fue la creación de `misistema.service`, ya que permitió ejecutar automáticamente nuestro programa al encender la Raspberry Pi, logrando un funcionamiento más autónomo sin necesidad de iniciarlo manualmente. Finalmente, la práctica me permitió entender la importancia de administrar correctamente los servicios y recursos de hardware para optimizar el sistema y evitar conflictos con otros programas.

García Olvera Emiliano.
Conclusión pendiente.

Hernández Alonso Antonio Adonahí.
Durante esta práctica comprendí que el proceso de arranque de la Raspberry Pi es un sistema multicapa perfectamente orquestado, donde cada etapa desde el bootloader en EEPROM hasta systemd cumple un propósito específico y verificable. Al inspeccionar el bootloader con `vcgencmd bootloader_version` y analizar los logs con `journalctl -b`, pude confirmar que los parámetros configurados en `config.txt` durante la Práctica 3 (I2C y SPI) no eran simples flags del sistema operativo, sino directivas procesadas por el firmware de la GPU en la Etapa 2, previo a la carga del kernel. El uso de `systemd-analyze blame` y `critical-chain` reveló que optimizar el arranque no consiste en deshabilitar el servicio más lento, sino en identificar y eliminar componentes de la cadena crítica que realmente retrasan la secuencia completa, como `lightdm` y el `graphical.target`, logrando una reducción considerable en el tiempo total. La creación del servicio `misistema.service` demostró cómo systemd permite automatizar la ejecución de programas embebidos desde el arranque, transformando el código de la Práctica 3 en un sistema autónomo que opera sin intervención manual, aunque también evidenció la importancia de gestionar correctamente los recursos GPIO, I2C y SPI mediante `systemctl stop` y `disable` para evitar conflictos en prácticas posteriores.

Jiménez Reyes Alan Yael.
En esta práctica me pareció interesante aprender qué es lo que realmente pasa dentro de la Raspberry Pi desde que la conectamos a la corriente hasta que está lista para usarse. Pude comprender de una forma mucho más aterrizada por qué modificamos la configuración del I2C y SPI en la práctica pasada, dándome cuenta de que esas instrucciones se leen desde los primeros pasos del encendido, mucho antes de que el sistema operativo siquiera inicie. Aunque ver todos los registros y tiempos de arranque parecía complicado al principio, lograr crear un servicio para que el programa de nuestro sensor combinado arrancara de forma automática al prender la placa fue una experiencia muy gratificante. Ver que ya no necesitaba conectarme por SSH desde mi computadora

para iniciar el código a mano me demostró cómo podemos crear proyectos verdaderamente independientes.

REFERENCIAS

[1] Lennart Poettering et al. (2024). “systemd System and Service Manager”, *systemd Documentation*. Disponible en: https://www.freedesktop.org/wiki/Software/systemd/

[2] Raspberry Pi Ltd. (2024). “Raspberry Pi Documentation — Boot Sequence”. Disponible en: https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

[3] The kernel development community. (2024). “Device Tree Usage”, *Linux Kernel Documentation*. Disponible en: https://www.kernel.org/doc/html/latest/devicetree/usage-model.html

[4] Lennart Poettering. (2024). “journalctl(1) - Linux man page”. Disponible en: https://man7.org/linux/man-pages/man1/journalctl.1.html